

# LLM 客户端开发实验报告

## LLM 客户端开发实验报告

### 实验目的

本实验旨在学习和掌握大型语言模型（LLM）API 的调用方法，实现基础的客户端交互功能。具体目标包括：

1. 掌握环境变量配置与加载方法
2. 实现 LLM API 的流式与非流式调用方式
3. 理解 HTTP/HTTPS 协议在 API 通信中的应用
4. 学习错误处理与异常捕获机制

### 实验内容

#### 1. 环境配置模块

实现了 `load_env()` 函数，从项目根目录的 `.env` 文件中读取配置参数：

- `BASE_URL`: LLM 服务接口地址
- `MODEL`: 模型名称
- `API_KEY`: 认证密钥
- `TEMPERATURE`: 温度参数（控制输出随机性）
- `MAX_TOKENS`: 最大生成 token 数

配置文件读取采用 UTF-8 编码，支持注释过滤（以 `#` 开头的行）。

#### 2. 非流式 LLM 客户端

`llm_client.py` 实现了同步调用模式：

- 使用 `http.client` 模块建立 HTTP/HTTPS 连接
- 解析 URL 提取主机名、路径和协议类型
- 构建符合 OpenAI API 规范的请求体
- 计算并输出性能指标：token 使用量、耗时、token 生成速度

3. 流式 LLM 客户端

chat\_client.py 实现了实时流式响应:

- 配置 stream: true 参数启用流式模式
- 通过循环逐块接收服务器响应
- 解析 data: 前缀的 SSE (Server-Sent Events) 格式
- 实时打印响应内容, 实现打字机效果
- 支持 SSL 连接, 配置证书验证选项

4. 错误处理机制

实现了多层错误捕获:

- 配置缺失检测: 检查必要环境变量
- HTTP 状态码验证: 处理 API 返回的错误状态
- JSON 解析异常: 捕获响应格式错误
- 连接异常处理: 捕获网络层错误

实验结果

功能验证

| 功能模块     | 测试结果 | 说明          |
|----------|------|-------------|
| 环境变量加载   | 通过   | 支持文件不存在检测   |
| 非流式调用    | 通过   | 正确返回响应及统计信息 |
| 流式调用     | 通过   | 实现实时逐字输出    |
| HTTPS 连接 | 通过   | 支持 SSL 证书配置 |
| 错误处理     | 通过   | 覆盖多种异常场景    |

输出示例

非流式客户端输出:

```
Base URL: http://localhost:1234/v1
Model: qwen/qwen3.5-2b
Prompt: 请用一句话介绍什么是LLM
=====
Response: 大型语言模型（LLM）是一种基于深度学习的人工智能系统，能够理解和生成人类语言。
=====
Token Usage: 45 tokens
Time Taken: 1.23 seconds
```

Speed: 36.59 tokens/second

### 流式客户端输出:

=== LLM 聊天客户端 ===

输入消息开始聊天, 按 **Ctrl+C** 退出

=====

你: 你好

助手: 您好! 我是一个AI助手, 很高兴为您服务。

## 实验总结

### 技术要点

1. **API 协议规范:** 遵循 OpenAI Chat Completions API 格式, 便于兼容多种 LLM 服务
2. **通信协议选择:** 支持 HTTP 和 HTTPS 两种协议, 根据 URL 自动选择连接方式
3. **响应处理策略:** 流式响应采用 SSE 协议, 非流式响应采用 JSON 格式
4. **配置管理:** 使用环境变量实现配置与代码分离, 提高安全性和可移植性

### 学习收获

通过本实验, 掌握了以下核心技能:

- LLM API 的基本调用方式和参数配置
- HTTP 客户端编程与响应处理
- 流式数据传输与实时输出技术
- 错误处理与异常捕获的最佳实践
- 环境变量管理与配置文件读取

### 改进方向

1. 增加请求超时设置, 避免长时间阻塞
2. 实现请求重试机制, 提高可靠性
3. 添加日志记录功能, 便于问题排查
4. 支持代理服务器配置, 适应内网环境